

All-In Academy

Full-Stack Texas Hold'em Training Lab

-Vijayender Kethi

1. Executive Summary

All-In Academy is a full-stack poker education platform that helps users practice live Texas Hold'em decisions, receive instant coaching feedback, and understand how poker hands are evaluated. The application was rebuilt from a single-file creative prototype into a professional software project with clear separation between UI, API, domain logic, validation, and tests.

The project is positioned for an engineering or games-related application because it combines interactive entertainment, real-time decision loops, backend game logic, and a coachable user experience. It demonstrates product thinking, system design, API design, state management, algorithmic reasoning, and practical testing.

What reviewers should notice	Evidence in project
Full-stack execution	React + TypeScript frontend communicates with a FastAPI backend over REST.
Product polish	Dashboard, simulator, learn area, coach panel, responsive styling, status indicators, and hand log.
Algorithmic component	Backend evaluates the best five-card hand from up to seven available cards using poker ranking categories and tiebreakers.
API discipline	Pydantic schemas validate action requests and coach requests; FastAPI exposes OpenAPI docs at /docs.
Testing mindset	pytest tests cover hand ranking comparison, wheel straight detection, and stage transitions; backend test run completed with 3 passing tests.
Interview readiness	Clear discussion points: tradeoffs, state flow, rules engine, limitations, deployment, and next upgrades.

2. Theme

Theme	How All-In Academy supports it
Entertainment technology	The app teaches through an interactive simulator rather than static content, turning learning into a playable experience.
Games and player decision loops	Each hand creates a repeatable loop: observe the state, choose an action, get a result, and learn from feedback.
Experimentation	The simulator can be extended into A/B tested coaching copy, training modes, or different opponent profiles.
Technical fundamentals	The backend includes algorithms, validation, API routes, game state, tests, and documented run instructions.
Ownership	The project shows the ability to take a rough prototype and turn it into a structured, maintainable product.

3. Product Overview

Item	Description
Product name	All-In Academy

Item	Description
One-line pitch	A full-stack Texas Hold'em training lab where users practice live poker decisions and get instant coaching feedback.
Primary user	Learners who understand basic cards but need practice making decisions under changing board states.
Core value	Transforms poker theory into interactive decisions with immediate feedback.
Differentiator	Combines educational content, a playable simulator, backend hand evaluation, and a coaching endpoint in one app.
Brand position	Professional, original, portfolio-safe training product with no dependency on third-party character IP.

4. User Experience and Product Flow

The frontend is designed around three main areas plus a persistent coach panel:

Overview: introduces the project, technology stack, API status, and product value.

Simulator: starts a new poker hand, displays player cards, opponent cards or card backs, community cards, pot, legal actions, hand log, and result.

Learn: provides educational reference content for poker concepts and hand ranking study.

Coach: accepts typed or quick-prompt questions and returns either general poker guidance or simulator-aware action advice.

4.1 Primary User Journey

Step	User action	System response
1	Open app	React app loads dashboard and checks backend health.
2	Launch simulator	Frontend calls POST /api/games and receives a fresh game state.
3	Review hand	UI renders hole cards, community slots, pot, stage, legal actions, and coach hint.
4	Choose action	Frontend sends action request to POST /api/games/{game_id}/actions.
5	Advance hand	Backend updates chips, pot, street, opponent response, log, and evaluation when needed.
6	Ask coach	Frontend sends prompt and game ID to POST /api/coach for contextual recommendation.
7	Showdown/result	Backend evaluates both players and returns winner, best hands, chip movement, and result text.

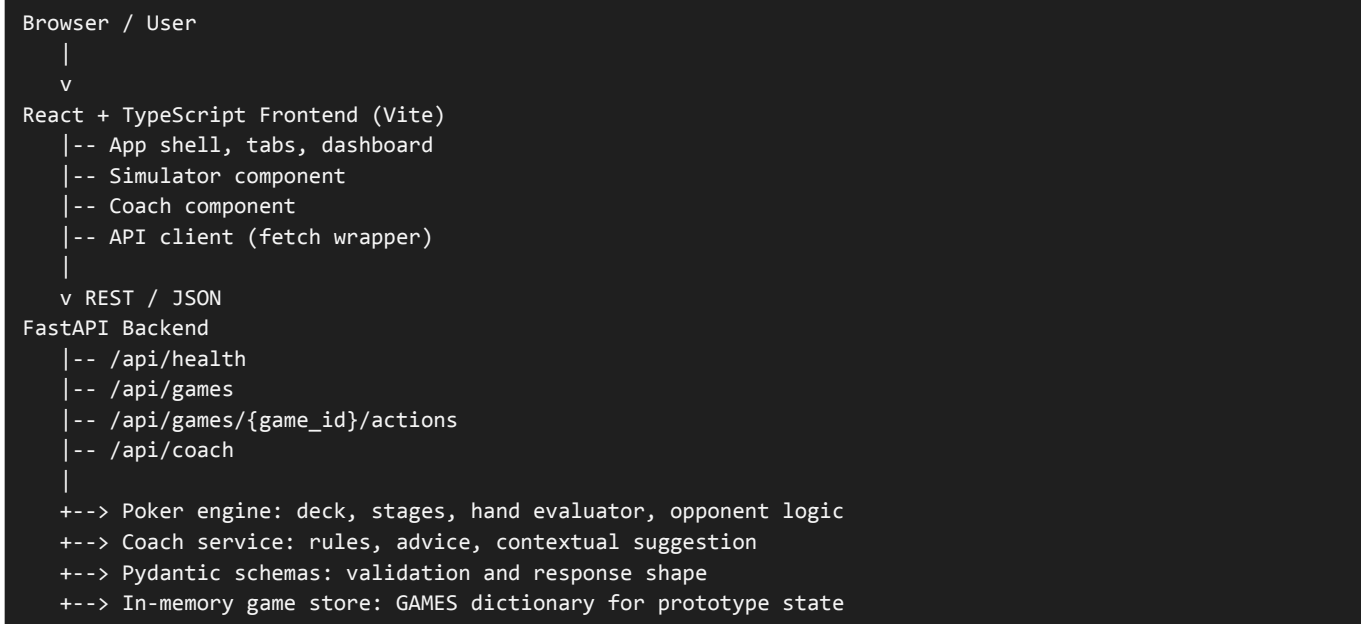
5. System Architecture

The architecture uses a clear split between presentation, API integration, backend routes, domain services, and validation. The frontend is responsible for rendering state and capturing user input. The backend is responsible for creating games, enforcing legal state transitions, calculating hand strength, making opponent decisions, and returning coaching advice.

Layer	Files	Responsibility
Frontend shell	frontend/src/App.tsx	Tabs, hero, API status, current game state, and layout composition.
Simulator UI	frontend/src/components/Simulator.tsx	Starts games, displays table, sends actions, renders legal actions, errors, and hand log.
Coach UI	frontend/src/components/Coach.tsx	Manages coach messages, quick prompts, form input, and coach API calls.
Reusable UI	CardView.tsx, MetricCard.tsx, Learn.tsx, styles.css	Card rendering, metric display, learning content, and visual design system.
API client	frontend/src/lib/api.ts	Centralized fetch wrapper and typed endpoint functions.
Backend routes	backend/app/main.py	FastAPI app, CORS, health route, game routes, action route, coach route.
Domain logic	backend/app/poker_engine.py	Deck, cards, game state, legal actions, stage advancement, hand evaluator, opponent logic.

Layer	Files	Responsibility
Coaching service	backend/app/coach.py	General poker guidance and contextual game-aware recommendations.
Validation	backend/app/schemas.py	Pydantic request and response models.
Tests	backend/tests/test_poker_engine.py	Unit tests for evaluator and game transition logic.

5.1 Architecture Diagram



6. Technology Stack

Area	Technology	Why it was selected
Frontend language	TypeScript	Adds type safety to card models, game states, legal actions, and coach messages.
Frontend framework	React	Good fit for changing UI state: tabs, game state, chat messages, loading, and errors.
Build tool	Vite	Fast local development and clean production build workflow.
Icons	lucide-react	Lightweight icon set for dashboard and action UI.
Backend framework	FastAPI	Modern Python API framework with automatic docs and Pydantic integration.
Validation	Pydantic	Enforces action kinds, amount bounds, and coach message constraints.
Testing	pytest	Simple Python test runner for poker engine correctness.
Local orchestration	docker-compose.yml	Runs backend and frontend containers together for local demonstration.

7. API Contract

The backend exposes a small REST API. FastAPI automatically generates interactive OpenAPI documentation at /docs when the backend server is running.

Endpoint	Method	Purpose	Key response / behavior
/api/health	GET	Health check	Returns status and service name so the frontend can show API online/offline.
/api/games	POST	Create game	Generates a UUID, creates a fresh game, stores it in memory, and returns public game state.

Endpoint	Method	Purpose	Key response / behavior
/api/games/{game_id}	GET	Read game	Returns the current state for a known game ID or 404 if missing.
/api/games/{game_id}/actions	POST	Apply player action	Validates action payload, updates game, stores updated state, and returns state.
/api/coach	POST	Ask coach	Returns a text reply and optional suggested action, using game context when available.

7.1 Request / Response Models

Model	Fields	Validation / purpose
ActionRequest	kind, amount	kind must be check_call, bet, or fold. amount is bounded from 0 to 10000.
CoachRequest	message, game_id	message is required and capped at 1000 characters. game_id is optional for contextual advice.
CoachResponse	reply, suggested_action	Returns the coach message and an optional recommended action.
GameState DTO	gameId, stage, cards, players, pot, log, legalActions, coachHint	Backend returns all data needed for rendering the simulator UI.

8. Backend Design Details

8.1 Game State

The backend represents a game with dataclasses: Card, Evaluation, PlayerState, and GameState. GameState stores the current street, remaining deck, community cards, both players, pot, call amount, minimum bet, dealer, hand number, result, best hands, and action log.

8.2 Stage Flow

Stage	Meaning	Transition trigger
PREFLOP	Players have two private cards; blinds are posted.	Check/call or bet action advances to FLOP unless someone folds.
FLOP	Three community cards are visible.	Next completed betting action advances to TURN.
TURN	Fourth community card is visible.	Next completed betting action advances to RIVER.
RIVER	Fifth community card is visible.	Next completed betting action triggers SHOWDOWN.
SHOWDOWN	Both hands are evaluated and winner is determined.	Hand is complete; new hand can be started.
COMPLETE	A player folded before showdown.	Hand is complete; new hand can be started.

8.3 Hand Evaluation Algorithm

The evaluator ranks poker hands by assigning each five-card hand a category and tiebreaker tuple. The backend then evaluates all five-card combinations from the available cards and selects the maximum sort key.

Category value	Hand name	Notes
9	Royal Flush	Ace-high straight flush.
8	Straight Flush	Five consecutive cards of same suit.
7	Four of a Kind	Quad rank plus kicker.
6	Full House	Trips rank plus pair rank.
5	Flush	Five same-suit cards ranked by high cards.
4	Straight	Includes wheel straight A-2-3-4-5.
3	Three of a Kind	Trips rank plus two kickers.
2	Two Pair	Top pair, second pair, kicker.
1	One Pair	Pair rank plus three kickers.
0	High Card	Five highest cards by value.

The current implementation is deterministic for test seeds and supports real card labels, suit colors, hidden opponent cards, and showdown reveal logic. It is efficient for a heads-up training simulator because evaluating combinations from seven cards is a small bounded computation.

8.4 Opponent and Coach Logic

Opponent decisions use a rule-based approximation of strength and pot pressure. The coach endpoint provides general education for topics like pot odds, bluffing, position, starting hands, and bankroll management. When a game ID is included, the coach uses the current game state to provide a simulator-aware recommendation such as Fold, Call, Check, Bet 50, or Bet 100.

9. Frontend Design Details

The React frontend is organized around reusable, typed components. App.tsx manages global UI state such as the active tab, backend health, and current game reference. Simulator.tsx owns game interaction. Coach.tsx owns chat state and submits prompts to the backend.

Component	State / props	Role
App	activeTab, apiStatus, currentGame	Composes the page, checks backend health, and passes current game context to Coach.
Simulator	game, loading, error; onGameChange prop	Starts games, sends player actions, renders table and decisions.
Coach	messages, input, loading; game prop	Provides coach chat and quick prompts with optional game ID.
CardView	card or hidden prop	Displays visible cards or card backs.
MetricCard	label, value, note	Reusable dashboard and simulator metric card.
Learn	static content	Supports learning-oriented presentation.
api.ts	request wrapper	Centralizes fetch calls and error handling.

9.1 Frontend Error Handling

The simulator catches backend failures and shows a user-facing message explaining that the FastAPI server must be running on port 8000. The top navigation also displays API status based on /api/health. This makes the demo easier to troubleshoot during review.

10. Testing and Validation

Backend tests were run in the project environment and completed successfully: 3 tests passed. The current tests focus on the highest-risk domain logic: hand comparison, straight detection, and game stage transitions.

Test	Purpose
test_royal_flush_beats_full_house	Confirms the evaluator ranks the strongest hand category above a full house.
test_wheel_straight_is_detected	Confirms A-2-3-4-5 is recognized as a straight.
test_game_advances_from_preflop_to_flop	Confirms a player action advances the game from PREFLOP to FLOP and deals three community cards.

```
cd backend
python -m pytest -q

# Result from this environment:
3 passed in 0.56s
```

11. Local Setup Instructions

Use these commands when running the project locally on Windows. Start the backend first, then open a second terminal for the frontend.

11.1 Backend - Windows Command Prompt

```
cd C:\Users\Vijay\Desktop\all_in_academy_fullstack\all_in_academy_fullstack\backend
python -m venv .venv
.venv\Scripts\activate.bat
pip install -r requirements.txt
fastapi dev app/main.py
```

11.2 Backend - Windows PowerShell

```
cd C:\Users\Vijay\Desktop\all_in_academy_fullstack\all_in_academy_fullstack\backend
python -m venv .venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
fastapi dev app/main.py
```

If PowerShell blocks activation, use: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

11.3 Frontend

```
cd C:\Users\Vijay\Desktop\all_in_academy_fullstack\all_in_academy_fullstack\frontend
npm install
npm run dev
```

Frontend URL: <http://localhost:5173>. Backend URL: <http://localhost:8000>. Backend API docs: <http://localhost:8000/docs>.

11.4 Docker Compose Option

```
cd all_in_academy_fullstack
docker compose up --build
```

The included docker-compose.yml runs a Python backend container and a Node frontend container for local demonstration. For production, separate Dockerfiles and a production build pipeline would be added.

12. Deployment Plan

For a professional submission, the best setup is to provide both a live demo and a GitHub repository. The frontend can be deployed as a static Vite build, while the backend should run as a hosted API service. The frontend should set `VITE_API_URL` to the deployed backend URL.

Part	Recommended approach	Notes
Frontend	Build with <code>npm run build</code> and deploy static files	Use a custom domain or clean project URL if possible.
Backend	Deploy FastAPI service behind HTTPS	Use a persistent host and update CORS to allow the frontend origin.
Environment	<code>VITE_API_URL</code>	Frontend reads this variable to call the correct backend base URL.
State	Move from in-memory GAMES to database/session storage	Necessary for multi-user production use and server restarts.
CI/CD	Add GitHub Actions	Run <code>pytest</code> and <code>npm build</code> on every push.

13. Security, Privacy, and Production Considerations

The current project is appropriate as a local prototype and portfolio demo. For production, the following areas should be improved:

Replace in-memory game storage with persistent database/session storage.

Add authentication if user history, accounts, or saved progress are introduced.

Restrict CORS to the deployed frontend domain instead of localhost development origins.

Add input rate limiting for the coach endpoint to prevent spam or abuse.

Add structured logging and request IDs for debugging.

Expand tests for all hand categories, ties, fold outcomes, betting limits, and API responses.

Add accessibility checks for keyboard navigation, color contrast, focus states, and screen-reader labels.

Add production Dockerfiles and a release pipeline.

14. Interview Talking Points

14.1 60-Second Pitch

I built All-In Academy, a full-stack poker training lab with a React and TypeScript frontend and a FastAPI backend. The app lets users play through a Texas Hold'em hand, make decisions, and ask an API-powered coach for advice based on the current game state. The backend handles the poker engine, hand evaluation, opponent decisions, Pydantic validation, and API routes. I built it to show not only UI design but also backend logic, state flow, testing, and product thinking.

14.2 Technical Tradeoffs to Discuss

Tradeoff	Current decision	Reasonable next step
Game persistence	In-memory GAMES dictionary	Move to Redis/PostgreSQL for multi-user deployment.
Coach intelligence	Rule-based coach	Add optional LLM integration or equity calculator behind the endpoint.
Opponent model	Simple strength/pot-pressure rules	Add difficulty levels and probabilistic behavior.
Testing scope	Core backend domain tests	Add API tests, frontend component tests, and CI.
Deployment	Local dev and docker-compose	Deploy frontend/backend separately with HTTPS and production CORS.

14.3 Resume / Application Bullets

Built a full-stack Texas Hold'em training platform with React, TypeScript, FastAPI, and Python, featuring a playable simulator and API-powered coaching panel.

Implemented a server-side poker hand evaluator that compares all five-card combinations from up to seven cards using ranking categories and tiebreaker tuples.

Designed REST endpoints with Pydantic validation for game creation, player actions, game retrieval, health checks, and contextual coach responses.

Created a reusable front-end component structure for cards, metrics, simulator state, coach messages, and learning content.

Added pytest coverage for core evaluator and game-transition logic, with the current backend test suite passing locally.

15. File Map

Path	Purpose
frontend/src/App.tsx	Top-level React shell, tabs, API status, and current game context.
frontend/src/components/Simulator.tsx	Playable table UI and player action handling.
frontend/src/components/Coach.tsx	Coach chat panel and quick prompts.
frontend/src/lib/api.ts	Typed API client for backend requests.
frontend/src/types.ts	Frontend TypeScript models for cards, players, evaluations, actions, and game state.
frontend/src/styles.css	Responsive visual system and table/coach styling.
backend/app/main.py	FastAPI app, routes, CORS, and in-memory game store.
backend/app/poker_engine.py	Deck, game state, evaluator, legal actions, opponent decisions, and coach hints.
backend/app/coach.py	Rule-based coaching logic and simulator-aware suggestions.
backend/app/schemas.py	Pydantic request and response models.
backend/tests/test_poker_engine.py	Core backend unit tests.
docker-compose.yml	Local two-service orchestration for backend and frontend.